

Building a Autoregressive Neural Network

Part 1

Luca WB

2026-01-28

Table of contents

0.1	Brief summary	2
0.2	Setup	2
0.3	Creating a baseline	2

0.1 Brief summary

In this post, we will implement an Autoregressive Neural Network from scratch, relying solely on the PyTorch tensor class. We assume prior familiarity with Neural Networks; however, if your knowledge feels a bit rusty or you need a refresher, I recommend reading this post beforehand [Building Neural Networks from Scratch](#).

The main reason for this is to learn how an Autoregressive NN works to generate words, for this, I'm drawing on Andrej Karpathy's video series about [makemore](#), a network capable of creating more words of the same type, so if you train with names, it generates more proper names it generates more words that remember proper names, and so on with anything that is formed by letters.

In this post, I will cover how to make a simple model for our baseline, and how to implement a model with MLP and compare them.

0.2 Setup

First, you need to download PyTorch and the dataset. For PyTorch, just download in the official site <https://pytorch.org/get-started/locally/>. Now, for the dataset, you can create your own with random names that you can think, but It's much easier just download the names.txt dataset from the Andrej repository <https://github.com/karpathy/makemore/blob/master/names.txt>.

0.3 Creating a baseline

In propose of this, it's just to create the most simple and naive model. It's important because we need some baseline to compare with our future models, so we will create a model called bigram, the logic is just to look to the last character. Note that you will use just one character of context for our model, and we will consider that the most small part

of our word is a character, for models like chatGPT, they don't use characters, they use combinations of characters similar to syllables.

So, to start, we need first import our dataset and PyTorch

```
1 import torch
2
3 # Basically makes a list of all the names
4 names = open("names.txt", "r").read().splitlines()
5 names[:5]
```

```
['emma', 'olivia', 'ava', 'isabella', 'sophia']
```

Most part of models usually can't handle with characters, so it's useful to convert this letters in numbers in some way. For this, there are many possibles, but I will use just a simple dictionary to convert them. But we

```
1 chars = sorted(list(set("".join(names)))) # Creates an ordered list with all letters in ou
2 charToInt = {s:i+1 for i,s in enumerate(chars)} # Creates a dict to convert chars to int,
3 charToInt["."] = 0 # I will explain later why we need a special character
4 print(charToInt)
```

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10, 'k': 11,
```

Just to get it ready, if we convert to int, so we can read it at the end, we will need an intToChar converter, so let's get it ready

```
1 intToChar = {s:i for i,s in charToInt.items()}
2 print(intToChar)
```

```
{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e', 6: 'f', 7: 'g', 8: 'h', 9: 'i', 10: 'j', 11: 'k',
```

Know, for our model, we need to calculate the total number that each sequence occurs, like, with we start with letter “a”, how many times occurs that “m” is the next character. And it's for this that we need and special characters, because we always need something to start, after all, the autoregressive model logic and take the output of the model and put it in its input, so we need an initial input. In our case, we will use “.” as the symbol to start a name/words and to stop word (without a final symbol, it would generate forever). To make more clear, see the code bellow

```

1 N = torch.zeros((27,27)).int()
2
3 for name in names:
4     chars = ["."] + list(name) + ["."] # turn the name in a list of characters and add "."
5     for ch1,ch2 in zip(chars, chars[1:]): # In each loop, pick up one letter in ch1, and t
6         id1, id2 = charToInt[ch1], charToInt[ch2]
7         N[id1, id2] += 1

```

Basically, this count how often some sequence of characters occurs, like the most common letter sequence is “n” follow by “.”, this mean, that the most commum letter to finish a name in our dataset it’s “n”. If you run with all the names, you can use the code bellow to find the most common occurrences

```

1 id1, id2 = (N == N.max()).nonzero(as_tuple=True) # Creates a boolean matrix that only it's
2 print(intToChar[id1.item()], "-->", intToChar[id2.item()], "occurs ", N.max().item())

```

n --> . occurs 6763

So let’s see how our bigrams are distributed

```

1 import matplotlib.pyplot as plt
2
3 plt.figure(figsize= (16,16))
4 plt.imshow(N, cmap="Blues")
5 for i in range(27):
6     for j in range(27):
7         chstr = intToChar[i] + intToChar[j]
8         plt.text(j,i, chstr, ha="center", va="bottom", color="gray")
9         plt.text(j,i, N[i,j].item(), ha="center", va="top", color="gray")
10
11 plt.axis("off")

```

ö	4713	1306	1542	1690	1531	417	669	874	591	2422	2963	1572	2538	1146	394	515	92	1639	2055	1308	78	376	307	134	535	929
a6640	aa556	ab541	ac470	ad1042	ae692	af134	ag168	ah2332	ai1650	aj175	ak568	al2528	am1634	an6438	ao63	ap82	aq60	ar3264	as1118	at687	au381	av834	aw161	ax182	ay2050	az435
b114	ba321	bb38	bc1	bd65	be655	bf0	bg0	bh41	bi217	bj1	bk0	bl103	bm0	bn4	bo105	bp0	bq0	br842	bs8	bt2	bu45	bv0	bw0	bx0	by83	bz0
c97	ca815	cb0	cc42	cd1	ce551	cf0	cg2	ch664	ci271	cj3	ck316	cl116	cm0	cn0	co380	cp1	cq11	cr76	cs5	ct35	cu35	cv0	cw0	cx3	cy104	cz4
d516	da1303	db1	dc3	dd149	de1283	df5	dg25	dh118	di674	dj9	dk3	dl60	dm30	dn31	do378	dp0	dq1	dr424	ds29	dt4	du92	dv17	dw23	dx0	dy317	dz1
e9953	ea679	eb121	ec153	ed384	ee1271	ef82	eg125	eh152	ei818	ej55	ek178	el3248	em769	en2675	eo269	ep83	eq14	er1958	es861	et580	eu69	ev463	ew50	ex132	ey1070	ez181
f80	fa242	fb0	fc0	fd0	fe123	ff44	fg1	fh1	fi160	fj0	fk2	fl20	fm0	fn4	fo60	fp0	fq0	fr114	fs6	ft18	fu10	fv0	fw4	fx0	fy14	fz2
g108	ga330	gb3	gc0	gd19	ge334	gf1	gg25	gh360	gi190	gj3	gk0	gl32	gm6	gn27	go83	gp0	gq0	gr201	gs30	gt31	gu85	gv1	gw26	gx0	gy31	gz1
h2409	ha2244	hb8	hc2	hd24	he674	hf2	hg2	hh1	hi729	hj9	hk29	hl185	hm117	hn138	ho287	hp1	hq1	hr204	hs31	ht71	hu166	hv39	hw10	hx0	hy213	hz20
i2489	ia2445	ib110	ic509	id440	ie1653	if101	ig428	ih95	ii82	ij76	ik445	il1345	im427	in2126	io588	ip53	iq52	ir849	is1316	it541	iu109	iv269	iw8	ix89	iy779	iz277
j71	ja1473	jb1	jc4	jd4	je440	jf0	ig0	ih45	ii119	ij2	jk2	jl9	jm5	jn2	jo479	jp1	jq0	jr11	js7	jt2	ju202	jv5	jw6	jx0	iy10	jz0
k363	ka1731	kb2	kc2	kd2	ke895	kf1	kg0	kh307	ki509	kj2	kk20	kl139	km9	kn26	ko344	kp0	kq0	kr109	ks95	kt17	ku50	kv2	kw34	kx0	ky379	kz2
l1314	la2623	lb52	lc25	ld138	le2921	lf22	lg6	lh19	li2480	lj6	lk24	ll1345	lm60	ln14	lo692	lp15	lq3	lr18	ls94	lt77	lu324	lv72	lw16	lx0	ly1588	lz10
m516	ma2590	mb112	mc51	md24	me818	mf1	mg0	mh5	mi1256	mj7	mk1	ml5	mm168	mn20	mo452	mp38	mq0	mr97	ms35	mt4	mu139	mv3	mw2	mx0	my287	mz11
n6763	na2977	nb8	nc13	nd704	ne1359	nf11	ng273	nh26	ni1725	nj58	nk195	nl195	nm19	nn1906	no496	np5	nq2	nr44	ns278	nt443	nu96	nv55	nw11	nx6	ny465	nz145
o855	oa149	ob140	oc114	od190	oe132	of34	og44	oh171	oi69	oj16	ok68	ol619	om261	on2411	oo115	op95	oq3	or1059	os504	ot118	ou275	ov176	ow114	ox45	oy103	oz54
p33	pa209	pb2	pc1	pd0	pe197	pf1	pg0	ph204	pi61	pj1	pk1	pl16	pm1	pn1	po59	pp39	pq0	pr151	ps16	pt17	pu4	pv0	pw0	px0	py12	pz0
q28	qa13	qb0	qc0	qd0	qe1	qf0	qg0	qh0	qi13	qj0	qk0	ql1	qm2	qn0	qo2	qp0	qq0	qr1	qs2	qt0	qu206	qv0	qw3	qx0	qy0	qz0
r1377	ra2356	rb41	rc99	rd187	re1697	rf9	rg76	rh121	ri3033	rj25	rk90	rl413	rm162	rn140	ro869	rp14	rq16	rr425	rs190	rt208	ru252	rv80	rw21	rx3	ry773	rz23
s1169	sa1201	sb21	sc60	sd9	se884	sf2	sg2	sh1285	si684	sj2	sk82	sl279	sm90	sn24	so531	sp51	sq1	sr55	ss461	st765	su185	sv14	sw24	sx0	sy215	sz10
t483	ta1027	tb1	tc17	td0	te716	tf2	tg2	th647	ti532	tj3	tk0	tl134	tm4	tn22	to667	tp0	tq0	tr352	ts35	tt374	tu78	tv15	tw11	tx2	ty341	tz105
u155	ua163	ub103	uc103	ud136	ue169	uf19	ug47	uh58	ui121	uj14	uk93	ul301	um154	un275	uo10	up16	uq10	ur414	us474	ut82	uu3	uv37	uw86	ux34	uy13	uz45
v88	va642	vb1	vc0	vd1	ve568	vf0	vg0	vh1	vi911	vj0	vk3	vl14	vm0	vn8	vo153	vp0	vq0	vr48	vs0	vt0	vu7	vv7	vw0	vx0	vy121	vz0
w51	wa280	wb1	wc0	wd8	we149	wf2	wg1	wh23	wi148	wj0	wk6	wl3	wm2	wn58	wo36	wp0	wq0	wr22	ws20	wt8	wu25	wv0	ww2	wx0	wy73	wz1
x164	xa103	xb1	xc4	xd5	xe36	xf3	xg0	xh1	xi102	xj0	xk0	xl31	xm1	xn1	xo41	xp0	xq0	xr0	xs31	xt70	xu5	xv0	xw3	xx38	xy30	xz19
y2007	ya2143	yb27	yc115	yd272	ye301	yf12	yg30	yh22	yi192	yj23	yk86	yl1104	ym148	yn1826	yo271	yp15	yq6	yr291	ys401	yt104	yu141	yv106	yw4	yx28	yy23	yz78
z160	za860	zb4	zc2	zd2	ze373	zf0	zg1	zh43	zi364	zj2	zk2	zl123	zm35	zn4	zo110	zp2	zq0	zr32	zs4	zt4	zu73	zv2	zw3	zx1	zy147	zz45

One thing very interesting you can note, it's that have many combinations that don't exist, like "bk" or "gc". This makes it impossible for our model to generate a name with this combination, it is ok to leave it like this, but it would be a good practice to add 1 in all values, thus ensuring that at least there is the minimal possibility of generating a rare sequence

1 $N = N + 1$

i	473	1307	1543	1691	1532	418	670	875	592	2423	2964	1573	2539	1147	395	516	93	r	1640	s	2056	t	1309	u	79	v	377	w	308	x	135	y	536	z	930				
a.6641	aa.557	ab.542	ac.471	ad.1043	ae.693	af.135	ag.169	ah.2333	ai.1651	aj.176	ak.569	al.2529	am.1635	an.8439	ao.64	ap.83	aq.61	ar.3265	as.1119	at.688	au.382	av.835	aw.162	ax.183	ay.2051	az.436													
b.115	ba.322	bb.39	bc.2	bd.66	be.656	bf.1	bg.1	bh.42	bi.218	bj.2	bk.1	bl.104	bm.1	bn.5	bo.106	bp.1	bq.1	br.843	bs.9	bt.3	bu.46	bv.1	bw.1	bx.1	by.84	bz.1													
c.98	ca.816	cb.1	cc.43	cd.2	ce.552	cf.1	cg.3	ch.665	ci.272	cj.4	ck.317	cl.117	cm.1	cn.1	co.381	cp.2	cq.12	cr.77	cs.6	ct.36	cu.36	cv.1	cw.1	cx.4	cy.105	cz.5													
d.517	da.1304	db.2	dc.4	dd.150	de.1284	df.6	dg.26	dh.119	di.675	dj.10	dk.4	dl.61	dm.31	dn.32	do.379	dp.1	dq.2	dr.425	ds.30	dt.5	du.93	dv.18	dw.24	dx.1	dy.318	dz.2													
e.3984	ea.680	eb.122	ec.154	ed.385	ee.1272	ef.83	eg.126	eh.153	ei.819	ej.56	ek.179	el.3249	em.770	en.2676	eo.270	ep.84	eq.15	er.1959	es.862	et.581	eu.70	ev.464	ew.51	ex.133	ey.1071	ez.182													
f.81	fa.243	fb.1	fc.1	fd.1	fe.124	ff.45	fg.2	fh.2	fi.161	fj.1	fk.3	fl.21	fm.1	fn.5	fo.61	fp.1	fq.1	fr.115	fs.7	ft.19	fu.11	fv.1	fw.5	fx.1	fy.15	fz.3													
g.109	ga.331	gb.4	gc.1	gd.20	ge.335	gf.2	gg.26	gh.361	gi.191	gj.4	gk.1	gl.33	gm.7	gn.28	go.84	gp.1	gq.1	gr.202	gs.31	gt.32	gu.86	gv.2	gw.27	gx.1	gy.32	gz.2													
h.2410	ha.2245	hb.9	hc.3	hd.25	he.675	hf.3	hg.3	hh.2	hi.730	hj.10	hk.30	hl.186	hm.118	hn.139	ho.288	hp.2	hq.2	hr.205	hs.32	ht.72	hu.167	hv.40	hw.11	hx.1	hy.214	hz.21													
i.2490	ia.2446	ib.111	ic.510	id.441	ie.1654	if.102	ig.429	ih.96	ii.83	ij.77	ik.446	il.1346	im.428	in.2127	io.589	ip.54	iq.53	ir.850	is.1317	it.542	iu.110	iv.270	iw.9	ix.90	iy.780	iz.278													
j.72	ja.1474	jb.2	jc.5	jd.5	je.441	jf.1	ig.1	jh.46	ji.120	jj.3	jk.3	jl.10	jm.6	jn.3	jo.480	jp.2	jq.1	jr.12	js.8	jt.3	ju.203	jv.6	jw.7	jx.1	jy.11	jz.1													
k.364	ka.1732	kb.3	kc.3	kd.3	ke.896	kf.2	kg.1	kh.308	ki.510	kj.3	kk.21	kl.140	km.10	kn.27	ko.345	kp.1	kq.1	kr.110	ks.96	kt.18	ku.51	kv.3	kw.35	kx.1	ky.380	kz.3													
l.1315	la.2624	lb.53	lc.26	ld.139	le.2922	lf.23	lg.7	lh.20	li.2481	lj.7	lk.25	ll.1346	lm.61	ln.15	lo.693	lp.16	lq.4	lr.19	ls.95	lt.78	lu.325	lv.73	lw.17	lx.1	ly.1589	lz.11													
m.517	ma.2591	mb.113	mc.52	md.25	me.819	mf.2	mg.1	mh.6	mi.1257	mj.8	mk.2	ml.6	mm.169	mn.21	mo.453	mp.39	mq.1	mr.98	ms.36	mt.5	mu.140	mv.4	mw.3	mx.1	my.288	mz.12													
n.6764	na.2978	nb.9	nc.214	nd.705	ne.1360	nf.12	ng.274	nh.27	ni.1726	nj.45	nk.59	nl.196	nm.20	nn.1907	no.497	np.6	nq.3	nr.45	ns.279	nt.444	nu.97	nv.56	nw.12	nx.7	ny.466	nz.146													
o.856	oa.150	ob.141	oc.115	od.191	oe.133	of.35	og.45	oh.172	oi.70	oj.17	ok.69	ol.620	om.262	on.2412	oo.116	op.96	oq.4	or.1060	os.505	ot.119	ou.276	ov.177	ow.115	ox.46	oy.104	oz.55													
p.34	pa.210	pb.3	pc.2	pd.1	pe.198	pf.2	pg.1	ph.205	pi.62	pj.2	pk.2	pl.17	pm.2	pn.2	po.60	pp.40	pq.1	pr.152	ps.17	pt.18	pu.5	pv.1	pw.1	px.1	py.13	pz.1													
q.29	qa.14	qb.1	qc.1	qd.1	qe.2	qf.1	qg.1	qh.1	qi.14	qj.1	qk.1	ql.2	qm.3	qn.1	qo.3	qp.1	qq.1	qr.2	qs.3	qt.1	qu.207	qv.1	qw.4	qx.1	qy.1	qz.1													
r.1378	ra.2357	rb.42	rc.100	rd.188	re.1698	rf.10	rg.77	rh.122	ri.3034	rj.26	rk.91	rl.414	rm.163	rn.141	ro.870	rp.15	rq.17	rr.426	rs.191	rt.209	ru.253	rv.81	rw.22	rx.4	ry.774	rz.24													
s.1170	sa.1202	sb.22	sc.61	sd.10	se.885	sf.3	sg.3	sh.1286	si.685	sj.3	sk.83	sl.280	sm.91	sn.25	so.532	sp.52	sq.2	sr.56	ss.462	st.766	su.186	sv.15	sw.25	sx.1	sy.216	sz.11													
t.484	ta.1028	tb.2	tc.18	td.1	te.717	tf.3	tg.3	th.648	ti.533	tj.4	tk.1	tl.135	tm.5	tn.23	to.668	tp.1	tq.1	tr.353	ts.36	tt.375	tu.79	tv.16	tw.12	tx.3	ty.342	tz.106													
u.156	ua.164	ub.104	uc.104	ud.137	ue.170	uf.20	ug.48	uh.59	ui.122	uj.15	uk.94	ul.302	um.155	un.276	uo.11	up.17	uq.11	ur.415	us.475	ut.83	uu.4	uv.38	uw.87	ux.35	uy.14	uz.46													
v.89	va.643	vb.2	vc.1	vd.2	ve.569	vf.1	vg.1	vh.2	vi.912	vj.1	vk.4	vl.15	vm.1	vn.9	vo.154	vp.1	vq.1	vr.49	vs.1	vt.1	vu.8	vv.8	vw.1	vx.1	vy.122	vz.1													
w.52	wa.281	wb.2	wc.1	wd.9	we.150	wf.3	wg.2	wh.24	wi.149	wj.1	wk.7	wl.14	wm.3	wn.59	wo.37	wp.1	wq.1	wr.23	ws.21	wt.9	wu.26	wv.1	ww.3	wx.1	wy.74	wz.2													
x.165	xa.104	xb.2	xc.5	xd.6	xe.37	xf.4	xg.1	xh.2	xi.103	xj.1	xk.1	xl.40	xm.2	xn.2	xo.42	xp.1	xq.1	xr.1	xs.32	xt.71	xu.6	xv.1	xw.4	xx.39	xy.31	xz.20													
y.2008	ya.2144	yb.28	yc.116	yd.273	ye.302	yf.13	yg.31	yh.23	yi.193	yj.24	yk.87	yl.1105	ym.149	yn.1827	yo.272	yp.16	yq.7	yr.292	ys.402	yt.105	yu.142	yv.107	yw.5	yx.29	yy.24	yz.79													
z.161	za.861	zb.5	zc.3	zd.3	ze.374	zf.1	zg.2	zh.44	zi.365	zj.3	zk.3	zl.124	zm.36	zn.5	zo.111	zp.3	zq.1	zr.33	zs.5	zt.5	zu.74	zv.3	zw.4	zx.2	zy.148	zz.46													

So, lets transform our probability matrix

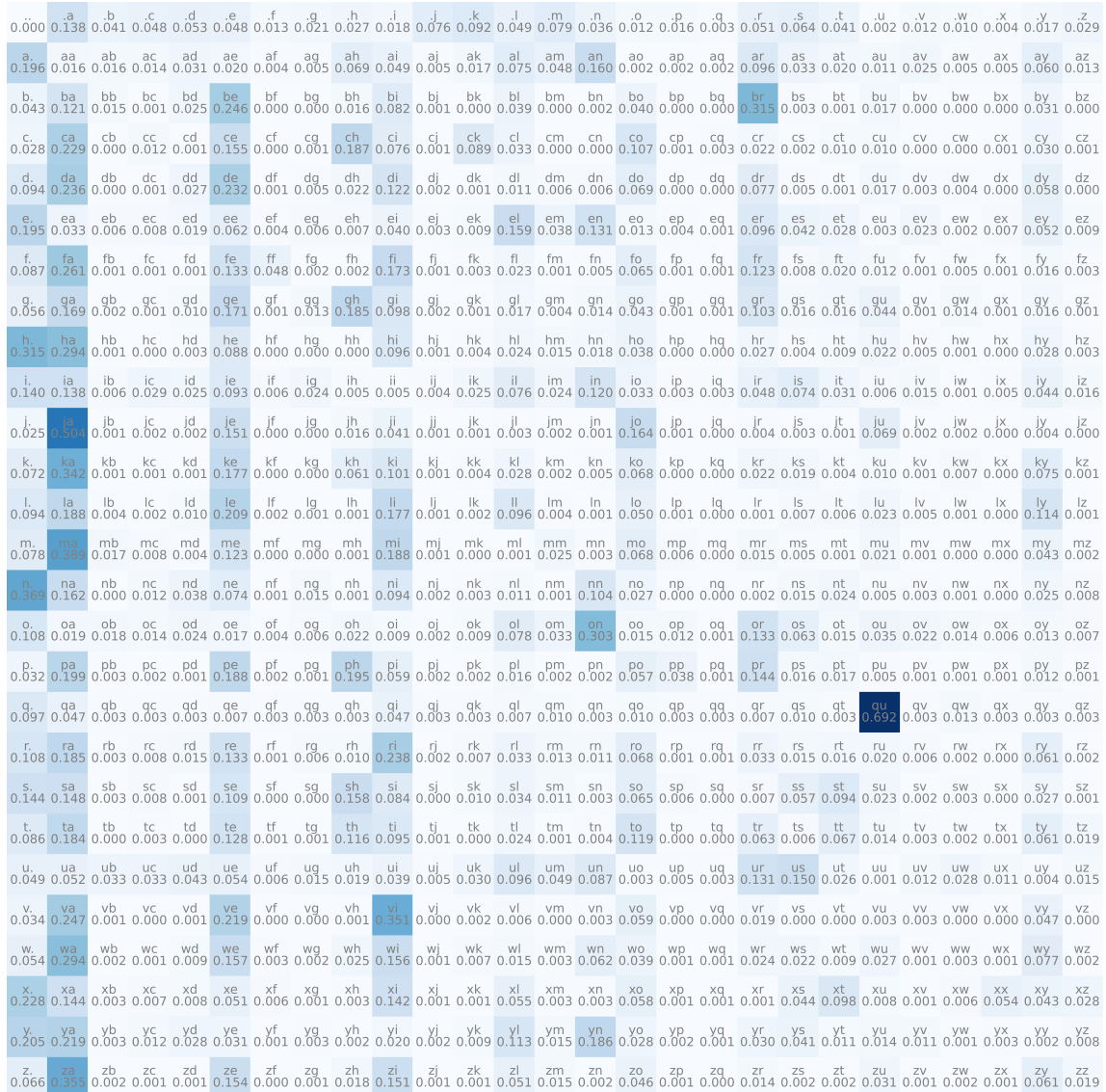
```
1 P = N
2 P = P / P.sum(dim=1, keepdims=True)
```

```
1 import matplotlib.pyplot as plt
2
3 plt.figure(figsize= (16,16))
4 plt.imshow(P, cmap="Blues")
5 for i in range(27):
6     for j in range(27):
7         chstr = intToChar[i] + intToChar[j]
8         plt.text(j,i, chstr, ha="center", va="bottom", color="gray")
```

```

9         plt.text(j,i, f"{P[i,j].item():.3f}", ha="center", va="top", color="gray")
10
11 plt.axis("off")

```



Some probabilities stay in 0 because the visualization it's limited to 3 decimal numbers. Now we already have our model, it's just our probability matrix P, bellow I will show how to use it.

```

1 import random
2
3 for i in range(10):
4     out = []

```

```

5     init = 0
6     while True:
7         id = torch.multinomial(P[init], num_samples=1, replacement=True).item()
8
9         if id == 0:
10            break
11
12        out.append(intToChar[id])
13        init = id
14    print("".join(out))

```

```

ladhai
ken
ile
chiliariali
jamitt
janany
h
sekal
trlelerilynemi
llsdrgaabr

```